
COGNIZANT — 50 HIGH-PRIORITY TECHNICAL QUESTIONS + ANSWERS

SECTION 1 — JAVA FUNDAMENTALS

Q1. What is Java?

Answer:

Java is a high-level, object-oriented, class-based programming language designed to be platform-independent.

Java follows the principle "**write once, run anywhere**" because Java source code is compiled into bytecode, which runs on the JVM.

Flow:

```
Java Code
  ↓
Compiler
  ↓
Bytecode
  ↓
JVM
  ↓
Machine Code
```

Q2. What is the difference between JDK, JRE and JVM?

Answer:

- **JVM** — executes Java bytecode.
- **JRE** — JVM + libraries required to run Java applications.
- **JDK** — JRE + development tools such as the Java compiler.

```
JDK
├── JRE
│   └── JVM
```

Easy memory trick:

JDK = Develop
JRE = Run
JVM = Execute

Q3. What are the four pillars of OOP?

Answer:

The four pillars are:

1. **Encapsulation** — wrapping data and methods together and controlling access to data.
2. **Inheritance** — allowing a child class to acquire properties and behavior from a parent class.
3. **Polymorphism** — one interface/name having different behavior, such as overloading and overriding.
4. **Abstraction** — hiding implementation details and exposing only necessary functionality.

Project example: A user object can encapsulate its data, different roles can inherit common behavior where appropriate, overridden methods provide runtime polymorphism, and interfaces can expose required operations without exposing implementation.

Recent Cognizant reports explicitly mention these four OOP concepts and project-based application of them. ([GeeksforGeeks](#))

Q4. What is method overloading vs method overriding?

Answer:

Overloading: Same method name with different parameter lists in the same class. It is compile-time polymorphism.

Overriding: A child class provides its own implementation of an inherited method with the same signature. It is runtime polymorphism.

```
// Overloading
add(int a, int b)
add(int a, int b, int c)
```

```
// Overriding
Animal → sound()
Dog → sound()
```

Q5. What is runtime polymorphism?

Answer:

Runtime polymorphism occurs when the method that gets executed is determined at runtime based on the **actual object**, rather than the reference type.

```
Animal a = new Dog();
a.sound();
```

If Dog overrides sound(), the Dog implementation executes.

This is achieved through **method overriding and dynamic method dispatch**.

Q6. What is the difference between == and .equals()?

Answer:

For objects:

- == compares **references**.
- .equals() compares **logical/content equality**, depending on how the class implements it.

For example:

```
String a = new String("Hello");
String b = new String("Hello");
a == b    // false
a.equals(b) // true
```

because they are different objects but contain the same text.

Q7. Why is String immutable in Java?

Answer:

A String object cannot be changed after it is created.

Immutability provides benefits such as:

- String pool optimization
- Security
- Thread safety
- Reliable hashing

For example:

```
String s = "Hello";
s.concat(" World");
```

doesn't modify the original "Hello" object; a new String is produced.

Q8. Difference between String, StringBuilder and StringBuffer?

Answer:

	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Performance for repeated modification	Poorer	Faster	Slower than StringBuilder
Thread-safe	Immutable	No	Yes, synchronized

Use:

- **String** → normal text that doesn't need repeated modification.
 - **StringBuilder** → repeated string modifications in a single-threaded context.
 - **StringBuffer** → when synchronized/thread-safe mutable string operations are required.
-

Q9. What is the difference between checked and unchecked exceptions?

Answer:

Checked exceptions are checked by the compiler and must be handled or declared.

Example: IOException, SQLException

Unchecked exceptions occur at runtime and are subclasses of RuntimeException.

Examples:

```
NullPointerException
ArithmeticException
ArrayIndexOutOfBoundsException
```

Q10. Difference between throw and throws?

Answer:

throw is used to **explicitly throw an exception**.

```
throw new IllegalArgumentException();
```

throws is used in a method declaration to **declare that the method may propagate an exception**.

```
void readFile() throws IOException
```

Q11. Difference between final, finally, and finalize()?

Answer:

- **final** → keyword used to restrict modification, inheritance, or overriding depending on where it is used.
- **finally** → block generally used for cleanup code after try/catch.
- **finalize()** → an old/deprecated mechanism associated with garbage collection; it should **not** be relied upon for resource cleanup.

For modern Java, use **try-with-resources** for resources such as files.

Q12. What is the difference between ArrayList and LinkedList?

Answer:

ArrayList is backed by a dynamic array and provides fast random access using an index.

LinkedList is based on linked nodes.

- ArrayList get(index) $\rightarrow O(1)$
- LinkedList get(index) $\rightarrow O(n)$
- Middle insertion/deletion $\rightarrow$ generally more favorable for LinkedList **once the position/node is known**
- ArrayList may require shifting elements for middle insertion/deletion.

Use **ArrayList** when random access is important.

Q13. What is HashSet and how does it prevent duplicates?

Answer:

HashSet stores unique elements using hashing.

When adding an object:

1. hashCode() helps determine where to look.
2. If a matching candidate exists, equals() checks equality.
3. If an equal element already exists, the new element isn't added.

Example:

```
HashSet<String> set = new HashSet<>();
```

```
set.add("Vikas");  
set.add("Java");  
set.add("Vikas");
```

Size = 2.

Q14. Difference between HashMap and HashSet?

Answer:

HashMap stores:

key $\rightarrow$ value

HashSet stores unique elements.

- HashMap keys are unique.
 - HashMap values can be duplicated.
 - HashMap allows one null key.
 - HashSet allows one null element.
 - HashSet is commonly implemented using a HashMap internally.
-

Q15. What is the difference between stack and heap memory in Java?

Answer:

The **stack** contains stack frames for method invocations, including local variables and execution state.

The **heap** contains objects and is managed by the Garbage Collector.

Example:

```
Student s = new Student();
```

Conceptually:

Stack:
s $\rightarrow$ reference

Heap:
Student object

Garbage Collection primarily reclaims **unreachable heap objects**.

SECTION 2 — SPRING / SPRING BOOT

These are especially important because your project claims Spring Boot, and recent Java/FSE-oriented Cognizant experiences explicitly mention Spring Boot, Spring concepts, dependency injection, MVC and related technologies.

Q16. What is Spring Boot?

Answer:

Spring Boot is a framework built on top of Spring that simplifies the development of Java applications by providing auto-configuration, starter dependencies, embedded servers, and production-ready features.

It reduces the amount of configuration required compared with traditional Spring applications.

Q17. Spring vs Spring Boot?

Answer:

Spring is the broader framework providing features such as dependency injection, MVC, AOP, and transaction management.

Spring Boot simplifies Spring application development through:

- Auto-configuration
- Starter dependencies
- Embedded servers
- Convention over configuration
- Easier application setup

Q18. What is Dependency Injection?

Answer:

Dependency Injection means an object receives the dependencies it needs from an external source instead of creating those dependencies itself.

Example:

```
@Service
class StudentService {

    private final StudentRepository repository;

    StudentService(StudentRepository repository) {
        this.repository = repository;
    }
}
```

Spring creates and injects the repository.

This reduces tight coupling and improves testability.

Q19. What is IoC?

Answer:

IoC means **Inversion of Control**.

Instead of our code manually creating and managing dependencies, the Spring container manages objects and their dependencies.

Dependency Injection is one way Spring implements IoC.

Q20. What are @Controller, @RestController, @Service, and @Repository?

Answer:

- @Controller → handles web/MVC requests, commonly returning views.
- @RestController → controller whose methods normally return response data such as JSON.
- @Service → represents business logic.
- @Repository → represents the persistence/data-access layer.

Typical architecture:

```
Client
↓
Controller
↓
Service
↓
Repository
↓
Database
```

Q21. What is @Autowired?

Answer:

@Autowired tells Spring to inject a matching dependency into a Spring-managed object.

However, **constructor injection is generally preferred** because dependencies become explicit and the object can be immutable.

Q22. What is Spring Security?

Answer:

Spring Security is a framework used to provide authentication and authorization for Spring applications.

It can handle:

- Authentication
- Authorization
- Password encoding
- Role-based access
- Security filters
- Protection against common web security problems

Q23. Explain the authentication flow in your project.

Answer:

A strong answer for your project:

When the user submits email and password, the request goes through Spring Security's authentication mechanism. The application loads the user using UserDetailsService, retrieves the stored password hash, and uses a password encoder such as BCrypt to verify the entered password. If authentication succeeds, the authenticated user becomes available through the security context. Authorization rules then determine which resources that user is allowed to access based on their role.

Q24. Authentication vs authorization?

Answer:

Authentication = Who are you?

Authorization = What are you allowed to access?

Example:

A student logs in with valid credentials.

Authentication verifies that the credentials belong to that user.

Authorization then determines whether that authenticated student can access a student endpoint.

If the student tries to access an admin-only resource:

Authentication → successful
Authorization → failed
Response → 403 Forbidden

Q25. Why is BCrypt used for passwords?

Answer:

BCrypt is a password-hashing algorithm designed for securely storing passwords.

Instead of storing the actual password, the application stores a BCrypt hash.

During login:

Entered password
↓
BCrypt verification
↓
Stored hash

If the password matches, authentication succeeds.

SECTION 3 — SQL / DBMS

SQL and project-specific database questions are consistently reported in Cognizant interviews. Recent 2026 reports specifically mention joins, keys, constraints, GROUP BY, ORDER BY, aggregate functions and DELETE/DROP/TRUNCATE. ([Glassdoor](#))

Q26. What is normalization?

Answer:

Normalization is the process of organizing database data to reduce redundancy and prevent anomalies such as:

- Insert anomaly
- Update anomaly
- Delete anomaly

It generally decomposes data into related tables based on dependencies.

Q27. Explain 1NF, 2NF and 3NF.

Answer:

1NF: Each cell contains a single atomic value.

2NF: Table is in 1NF and has no partial dependency on part of a composite key.

3NF: Table is in 2NF and has no transitive dependency where a non-key attribute depends on another non-key attribute.

Memory:

1NF → Atomic values
2NF → No partial dependency
3NF → No transitive dependency

Q28. Primary key vs foreign key?

Answer:

A **primary key** uniquely identifies a row and cannot contain NULL values.

A **foreign key** references a key in another table and establishes a relationship between tables.

Example:

Branch
branch_id ← Primary Key

Student
student_id ← Primary Key
branch_id ← Foreign Key

Multiple students can have the same branch_id.

Q29. What is the difference between INNER JOIN and LEFT JOIN?

Answer:

INNER JOIN returns only matching rows from both tables.

LEFT JOIN returns **all rows from the left table**, plus matching rows from the right table. If no match exists, right-side columns are NULL.

Example:

```
SELECT s.student_name, b.branch_name
FROM Student s
LEFT JOIN Branch b
ON s.branch_id = b.branch_id;
```

Use LEFT JOIN when you must preserve all students even if some don't have a matching branch.

Q30. DELETE vs TRUNCATE vs DROP?

Answer:

Command	Removes	WHERE	Structure
DELETE	Rows	Yes	Remains
TRUNCATE	All rows	No	Remains
DROP	Table	No	Removed

DELETE is useful when selectively removing rows.

TRUNCATE removes all rows.

DROP removes the table itself.

Transaction behavior can depend on the database system, so don't claim a universal rollback rule across all databases.

Q31. What is an index?

Answer:

An index is a database data structure that helps the database find rows more efficiently without scanning the entire table.

For example:

```
SELECT *
FROM Student
WHERE email = 'vikas@example.com';
```

If email has an appropriate index, the database can locate matching rows more efficiently.

The trade-off is that indexes consume storage and can make inserts/updates/deletes more expensive because the index must also be maintained.

Q32. What is GROUP BY and HAVING?

Answer:

GROUP BY groups rows based on one or more columns, usually for aggregate calculations.

HAVING filters the **groups after aggregation**.

Example:

```
SELECT department, COUNT(*)
FROM Employee
GROUP BY department
HAVING COUNT(*) > 5;
```

WHERE filters rows before grouping.

HAVING filters groups after grouping.

Q33. What are ACID properties?

Answer:

ACID describes reliable database transactions:

- **Atomicity** — transaction happens completely or not at all.
- **Consistency** — database moves from one valid state to another.
- **Isolation** — concurrent transactions should not improperly interfere.
- **Durability** — committed changes persist even after failures.

SECTION 4 — OPERATING SYSTEM

Q34. Process vs thread?

Answer:

A **process** is a program in execution with its own virtual address space and resources.

A **thread** is an execution unit within a process.

Threads of the same process share resources such as code, data and heap, while each thread has its own stack, registers and program counter.

Threads are generally cheaper to create and switch than processes.

Q35. What is context switching?

Answer:

Context switching occurs when the CPU switches execution from one process/thread to another.

The OS saves the current execution context and restores the saved context of the next task.

It has overhead because the CPU spends time saving/restoring state rather than executing useful application instructions, and switching can also affect caches/TLBs.

Q36. What is deadlock?

Answer:

Deadlock is a situation where processes or threads are permanently blocked because each is waiting for a resource held by another.

Four necessary conditions:

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

Example:

A holds Resource 1 → waits for Resource 2
B holds Resource 2 → waits for Resource 1

Neither can proceed.

Q37. What is virtual memory and paging?

Answer:

Virtual memory allows a process to use a virtual address space larger than the available physical memory.

Paging divides virtual memory into fixed-size pages and physical memory into frames.

When a required page isn't currently in physical memory, a **page fault** occurs and the OS must bring the page into memory.

SECTION 5 — COMPUTER NETWORKS

You haven't studied CN yet, so these are the **minimum interview-level questions** you should learn.

Q38. TCP vs UDP?

Answer:

TCP:

- Connection-oriented
- Reliable
- Ordered delivery
- Error/flow/congestion control
- Higher overhead

UDP:

- Connectionless
- No guarantee of delivery/order
- Lower overhead
- Faster for applications where low latency matters

Examples:

- TCP → HTTP/HTTPS, file transfer
 - UDP → DNS queries, streaming/real-time applications
-

Q39. HTTP vs HTTPS?

Answer:

HTTP transfers web data without TLS encryption.

HTTPS is HTTP over **TLS**, providing encryption and server authentication and helping protect data from interception/tampering.

HTTPS normally uses port **443**.

HTTP normally uses port **80**.

Q40. What is an IP address?

Answer:

An IP address identifies a network interface/device address at the Internet Protocol layer so packets can be routed between networks.

Two common versions are:

- IPv4 → 32-bit
- IPv6 → 128-bit

Example IPv4:

192.168.1.10

Q41. What is DNS?

Answer:

DNS stands for **Domain Name System**.

It translates domain names into IP addresses.

For example:

google.com
↓
DNS
↓
IP address

This allows users to use human-readable domain names instead of remembering IP addresses.

Q42. What is the difference between GET and POST?

Answer:

GET is generally used to retrieve resources.

POST is generally used to submit data to the server, often resulting in creation or another state-changing operation.

GET parameters are commonly placed in the URL/query string, while POST data is commonly placed in the request body.

For APIs, use the HTTP method according to the intended semantics rather than simply thinking "GET = safe, POST = secure."

SECTION 6 — DSA / CODING

Recent Cognizant experiences report basic array/string coding, duplicate removal, palindrome/reversal, searching/sorting, and live explanation of coding logic. ([GeeksforGeeks](https://www.geeksforgeeks.org/))

Q43. How do you reverse a String?

Answer:

Using StringBuilder:

```
String str = "hello";
```

```
String reversed = new StringBuilder(str)
    .reverse()
    .toString();
```

Time complexity: **O(n)**.

Q44. How do you check whether a String is a palindrome?

Answer:

Compare characters from both ends using two pointers.

left → ← right

If every pair matches, it is a palindrome.

Time: **O(n)**

Space: **O(1)** if done in-place on the character sequence.

Q45. How do you remove duplicates from an array?

Answer:

For a simple Java solution, use a HashSet.

```
Set<Integer> set = new HashSet<>();
```

```
for (int x : arr) {
    set.add(x);
}
```

A HashSet keeps only unique values.

Average insertion: **O(1)**.

Overall average: **O(n)**.

Q46. Explain Kadane's Algorithm.

Answer:

Kadane's algorithm finds the maximum subarray sum in **O(n)** time and **O(1)** extra space.

At every element:

```
currentSum =  
max(current element,  
    currentSum + current element)
```

Then:

```
maxSum = max(maxSum, currentSum)
```

Example:

```
[-2,1,-3,4,-1,2,1,-5,4]
```

Maximum sum:

```
4 + (-1) + 2 + 1 = 6
```

SECTION 7 — YOUR PROJECT

This is the section I would drill hardest in your actual interview because your resume claims backend/authentication work and Cognizant interview reports consistently show substantial project-specific questioning. ([Glassdoor](#))

Q47. Explain your project.

Answer:

My project is an Academic Project Approval System designed to replace the manual project submission and approval process in colleges. The system manages project submission and approval digitally and provides role-based access for users such as students, faculty/guides, HOD or administrators, depending on the configured roles. Students can work on faculty-proposed projects or submit their own projects for approval. The backend was developed using Java and Spring Boot, with MySQL as the database and HTML, CSS and JavaScript for the frontend. My main contribution was backend development, particularly authentication and session/security-related functionality.

Important: Use the **actual roles in your implemented application**. Your previous explanations have alternated between Student/Faculty/HOD/Admin and Student/Guide/Branch Admin/Super Admin. Fix this inconsistency before the interview.

Q48. Explain the complete authentication and authorization flow in your project.

Answer:

The user provides credentials such as email and password. Spring Security processes the authentication request. The application loads the user from the database using the configured user-details mechanism. The stored BCrypt password hash is then used to verify the entered password. If authentication succeeds, the authenticated principal is available through the security context. After authentication, authorization rules check the user's role and determine which endpoints or resources the user is allowed to access. Therefore, authentication answers "Who is the user?" while authorization answers "What is the user allowed to do?"

Q49. Why should you not trust studentId sent by the client?

Answer: Because the client is **untrusted input**.

If the frontend sends:

```
{  
  "studentId": 25  
}
```

the backend should not blindly assume that the authenticated user is student 25.

Otherwise, a malicious user could change:

```
studentId = 25
```

```
to: studentId = 26
```

and potentially perform actions on another student's data.

The backend should derive the current user's identity from the **authenticated security context/principal** and use that server-side identity when accessing protected student resources.

This is one of the most important answers you were missing in the mock interview.

Q50. 401 vs 403?

401 Unauthorized means the request has not been successfully authenticated or valid authentication credentials are missing/invalid.

403 Forbidden means the user is authenticated but **does not have permission** to access the requested resource.

Example:

No valid login
→ 401

Logged-in student tries admin endpoint
→ 403

